Python Code Documentation

July 2, 2026

1 Overview

This module simulates the planar kinematics of a Roberts-linkage-type mechanism built and verified in the *Linkage* software, matching a specific six-link topology found directly from the user's Linkage build (not assumed or guessed):

Link	Connects
1	$B \to D$
2	$A \to C$
3	$D \to P$
4	$C \to P$
5	$D \to L$
6	$C \to L$

The two fixed pivots A, B each drive a crank (AC, BD). Points P and L are each the apex of their own isosceles-triangle-like construction on the *same* base points C, D — but C and D are never directly linked to one another. This is the single most important structural fact the module is built around, so it is worth stating plainly:

> **P and L are computed completely independently of one another.** There is no rigid body that contains both of them. P is pinned to C, D by lengths CP, DP; L is pinned to C, D by its own, different lengths CL, DL. Moving C, D moves both P and L, but P and L do not move rigidly *with each other*.

A degree-of-freedom count via Gruebler's equation confirms why both A and B must be driven (matching the real file's rpm $= +15$ / rpm $= -15$ pair):

$$N = 7 \text{ (ground, } AC, BD, CP, DP, CL, DL),$$

$$J = 8 \text{ (A, B, P, L each 1 joint; C, D each 2, since 3 bodies meet)}$$

$$\text{DOF} = 3(N-1) - 2J = 3(6) - 2(8) = 18 - 16 = 2.$$

Two genuine degrees of freedom. The module's driving convention adds the *same* angle θ to each pivot's own rest angle ($\phi_{0,A} + \theta$ and $\phi_{0,B} + \theta$), which reproduces the observed $+15/-15$ rpm pair because $\phi_{0,A}$ and $\phi_{0,B}$ are themselves mirror-image angles.

2 Module-Level Setup

```
OUTPUT_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "output")
os.makedirs(OUTPUT_DIR, exist_ok=True)
```

`OUTPUT_DIR` is anchored to the script's *own* location (`__file__`), not the current working directory the script happens to be launched from. This was a real, previously-fixed bug: a bare relative path (`"output/..."`) fails with `FileNotFoundError` if the script is run from a different directory (e.g. `d:\IUCAA` on Windows), since matplotlib does not create missing directories itself.

3 Section 1 — Geometry Construction

3.1 `geometry_defaults()`

```
def geometry_defaults():
    A = np.array([-265.0, 0.0])
    B = np.array([265.0, 0.0])
    C = np.array([-132.5, -254.96])
    D = np.array([132.5, -254.96])
    P = np.array([0.0, 0.0])
    L = np.array([0.0, -29.2])
    return A, B, C, D, P, L
```

Purpose. Returns the exact rest-position coordinates read directly from the user's Linkage file: $A = (-265, 0)$, $B = (265, 0)$ (fixed pivots, half-width $W = 265\,\text{mm}$); $C = (-132.5, -254.96)$, $D = (132.5, -254.96)$ (base points, depth $q = 254.96\,\text{mm}$); $P = (0, 0)$ (pinned exactly on the A–B line, per the corrected reading of the Roberts-linkage paper's Fig. 1); $L = (0, -29.2)$ (load point, 29.2 mm below P).

Returns. A 6-tuple of length-2 `numpy` arrays, `(A, B, C, D, P, L)`, in millimetres.

No parameters. This is the fixed, "known good" reference geometry every other function is checked against.

3.2 `geometry_at_base_depth(W, q, L0, isosceles="apex_at_C")`

```
def geometry_at_base_depth(W, q, L0, isosceles="apex_at_C"):
    if isosceles != "apex_at_C":
        raise ValueError("Only 'apex_at_C' is implemented in this closed form."
            )
    c_val = W / 2.0
    A = np.array([-W, 0.0])
    B = np.array([W, 0.0])
    C = np.array([-c_val, -q])
    D = np.array([c_val, -q])
    P = np.array([0.0, 0.0])
    L = np.array([0.0, float(np.asarray(L0)[1])])
    return A, B, C, D, P, L
```

Purpose. A parametrised, *fast* version of `geometry_defaults()`, used everywhere the base geometry needs to vary (the interactive explorer's q/W sliders, and the stability-crossing search). It builds the rest geometry for an arbitrary half-width W, base depth q, and load-point depth (taken from the y-component of `L0`), while keeping A, B, P constructed the same way as `geometry_defaults()`:

P pinned exactly to the origin, and C, D satisfying the "apex_at_C" isosceles/congruence condition ($AC = CP$).

The key mathematical shortcut. Under the apex_at_C condition, with P pinned at the origin, it was shown (symbolically, in an earlier version of this module) that the x-coordinate of C and D collapses to the closed form

$$c = \frac{W}{2} \qquad \textit{identically, for any } \; q.$$

This is not a coincidence needing re-derivation each call — it is a fixed algebraic identity of this particular congruence choice. Using it directly (instead of a symbolic `sympy.solve()` call, as an earlier version of this module did) makes this function cheap enough to call tens of thousands of times inside a search loop, which is exactly what Sections 7–8 do.

Parameters.

- `W` (float): half-width of A, B, mm.
- `q` (float): depth of C, D below the A–B line, mm.
- `L0` (array-like, length 2): only the y-component is used — L is always placed on the centerline $x = 0$, matching the mirror-symmetric construction used throughout.
- `isosceles` (str): currently must be `"apex_at_C"` (the branch matching the real Linkage build); any other value raises `ValueError` rather than silently doing the wrong thing.

Returns. Same 6-tuple format as `geometry_defaults()`.

4 Section 2 — Forward Kinematics

4.1 Class `RobertsLinkageCLD`

Encapsulates one linkage instance: fixed link lengths derived once from a rest configuration, plus methods to compute the full configuration (C, D, P, L) at any drive angle θ.

4.1.1 `__init__(self, A, B, C0, D0, P0, L0)`

```
def __init__(self, A, B, C0, D0, P0, L0):
    self.A, self.B = A, B
    self.C0, self.D0, self.P0, self.L0 = C0, D0, P0, L0
    self.AC = np.linalg.norm(C0 - A)
    self.BD = np.linalg.norm(D0 - B)
    self.CP = np.linalg.norm(P0 - C0)
    self.DP = np.linalg.norm(P0 - D0)
    self.CL = np.linalg.norm(L0 - C0)
    self.DL = np.linalg.norm(L0 - D0)
    self.phi0_A = self._angle_from_vertical(C0 - A)
    self.phi0_B = self._angle_from_vertical(D0 - B)
```

Purpose. Given a single rest configuration (all six points), derives and stores the six fixed rigid-link lengths (AC, BD, CP, DP, CL, DL) and the two rest angles ($\phi_{0,A}, \phi_{0,B}$, each measured from vertical at A and B respectively). These lengths are fixed for the lifetime of the object — every later call to `configuration(theta)` reuses them rather than re-deriving anything from the rest points, which is what makes it a valid rigid-link model.

Parameters. `A, B, C0, D0, P0, L0`: length-2 arrays, the rest-position coordinates (typically produced by `geometry_defaults()` or `geometry_at_base_depth()`).

4.1.2 `_angle_from_vertical(vec)` (static method)

```
@staticmethod
def _angle_from_vertical(vec):
    down = np.array([0.0, -1.0])
    cosang = np.dot(vec, down) / np.linalg.norm(vec)
    ang = np.arccos(np.clip(cosang, -1, 1))
    sign = np.sign(vec[0]) if vec[0] != 0 else 1
    return sign * ang
```

Purpose. Computes the signed angle between a vector and straight down $(0, -1)$, using the vector's x-component to assign the sign (positive = tilted to the right of vertical, negative = left). Used to find each crank arm's rest angle ($\phi_{0,A} = \angle(C_0 - A)$, $\phi_{0,B} = \angle(D_0 - B)$), which is the reference angle that θ is added to during motion.

`np.clip(cosang, -1, 1)` guards against `arccos` domain errors from floating-point values that drift fractionally outside $[-1, 1]$.

4.1.3 `_get_C(self, theta)` and `_get_D(self, theta)`

```
def _get_C(self, theta):
    ang = self.phi0_A + theta
    return self.A + self.AC * np.array([np.sin(ang), -np.cos(ang)])

def _get_D(self, theta):
    ang = self.phi0_B + theta
    return self.B + self.BD * np.array([np.sin(ang), -np.cos(ang)])
```

Purpose. Directly places C and D by rotating each crank arm (length AC or BD) by the drive angle θ away from its rest angle. No iterative solving is needed here — C and D are each a simple function of a single independent input.

The critical modelling decision: both functions add the *same* θ to their own (different-signed) rest angle. Since $\phi_{0,A}$ and $\phi_{0,B}$ are mirror-image angles ($\phi_{0,B} \approx -\phi_{0,A}$ for this symmetric geometry), adding the same θ to both reproduces a genuinely counter-rotating pair of cranks in absolute terms — exactly matching the real Linkage file's rpm = +15 at A and rpm = −15 at B. This was verified directly: the alternative (mirrored) convention, $\phi_{0,B} - \theta$, was tested and found to move P, L *only* along the vertical centerline with no horizontal excursion at all — inconsistent with the real, observed horizontal coupler-curve traces.

4.1.4 `_circle_intersect(C, D, rC, rD, ref)` (static method)

```
@staticmethod
def _circle_intersect(C, D, rC, rD, ref):
    d = np.linalg.norm(D - C)
    if d > rC + rD or d < abs(rC - rD) or d == 0:
        return None
    a = (rC**2 - rD**2 + d**2) / (2 * d)
    h2 = rC**2 - a**2
    if h2 < 0:
```

```
        return None
    h = np.sqrt(h2)
    mid = C + a * (D - C) / d
    perp = np.array([-(D - C)[1], (D - C)[0]]) / d
    X1, X2 = mid + h * perp, mid - h * perp
    return X1 if np.linalg.norm(X1 - ref) < np.linalg.norm(X2 - ref) else X2
```

Purpose. The core geometric primitive of the whole module — finds the intersection of two circles, circle(C, r_C) and circle(D, r_D), using the standard two-circle intersection formula:

$$a = \frac{r_C^2 - r_D^2 + d^2}{2d}, \qquad h = \sqrt{r_C^2 - a^2}, \qquad d = \|D - C\|,$$

with the midpoint $\text{mid} = C + a\,\hat{u}$ (where $\hat{u}$ is the unit vector from C to D) and the two solutions offset by $\pm h$ along the perpendicular direction.

Degeneracy guards. Returns `None` if the circles do not intersect at all ($d > r_C + r_D$, one circle inside the other without touching, or $d = 0$, i.e. C and D coincide) — this happens near the mechanism's physical limits of travel and is handled gracefully rather than raising an exception mid-sweep.

Branch selection. Two circles generically intersect at *two* points, X_1 and X_2. The function returns whichever is closer to a supplied reference point `ref` (always the *rest*-position value, P_0 or L_0). This picks the branch that is continuous with the mechanism's assembled rest configuration, rather than the mechanism "flipping" to its mirror assembly mode as θ sweeps. This was checked directly: the two candidate solutions stay well separated (hundreds of mm apart) near $\theta = 0$ for this geometry, so the nearest-branch selection is robust, not borderline.

4.1.5 `configuration(self, theta)`

```
def configuration(self, theta):
    C = self._get_C(theta)
    D = self._get_D(theta)
    P = self._circle_intersect(C, D, self.CP, self.DP, self.P0)
    L = self._circle_intersect(C, D, self.CL, self.DL, self.L0)
    return C, D, P, L
```

Purpose. The single public entry point for full forward kinematics: given a drive angle θ (radians), returns the complete mechanism configuration (C, D, P, L).

This is where the topology fix lives. P and L are each computed by an independent call to `_circle_intersect`, using the freshly-computed C, D as the shared base — *not* by carrying L rigidly through a coordinate frame anchored to P (which is what an earlier, structurally incorrect version of this module did, implicitly assuming a rigid C–P–D–L body that does not exist in the real mechanism).

5 Section 3 — Trace Generation

5.1 `trace_point(linkage, which, theta_deg_range=5.0, npts=201)`

```
def trace_point(linkage, which, theta_deg_range=5.0, npts=201):
    idx = {"C": 0, "D": 1, "P": 2, "L": 3}[which]
    thetas = np.radians(np.linspace(-theta_deg_range, theta_deg_range, npts))
    pts = []
```

```
    for t in thetas:
        cfg = linkage.configuration(t)
        p = cfg[idx]
        if p is not None:
            pts.append(p)
    return np.array(pts)
```

Purpose. Sweeps θ linearly over $[-\theta_{\text{range}}, +\theta_{\text{range}}]$ (in degrees, converted to radians internally) and collects the resulting (x, y) positions of whichever point is requested (`"C"`, `"D"`, `"P"`, or `"L"`) into an $(n, 2)$ `numpy` array — this is the "coupler curve" the rest of the module analyses.

Parameters.

- `linkage`: a `RobertsLinkageCLD` instance.
- `which`: one of `"C"`, `"D"`, `"P"`, `"L"` — selects which of the four configuration outputs to trace.
- `theta_deg_range`: half-width of the sweep, in degrees.
- `npts`: number of samples across the sweep.

Degenerate points are silently skipped (any θ for which `configuration` returns `None` for the requested point is simply not appended), so the returned array can be shorter than `npts` if part of the sweep goes out of the mechanism's valid range of motion.

6 Section 4 — Flatness / Stability Diagnostics

6.1 `flatness_rms(pts)`

```
def flatness_rms(pts):
    if len(pts) < 3:
        return np.inf
    m, c = np.polyfit(pts[:, 0], pts[:, 1], 1)
    resid = pts[:, 1] - (m * pts[:, 0] + c)
    return np.sqrt(np.mean(resid**2))
```

Purpose. Quantifies how "flat" a traced curve is: fits the best straight line $y = mx + c$ through the points (least-squares) and returns the RMS deviation of the actual trace from that line. Used to check the paper's central claim that point P traces (nearly) flat.

6.2 `curvature_sign(pts)`

```
def curvature_sign(pts):
    if len(pts) < 3:
        return np.nan
    x, y = pts[:, 0], pts[:, 1]
    xc = x - x.mean()
    a, b, c = np.polyfit(xc, y, 2)
    return a
```

Purpose. Fits a quadratic $y = a(x - \bar{x})^2 + b(x - \bar{x}) + c$ to the trace (centering x on its own mean first, purely for numerical conditioning of the least-squares fit) and returns the quadratic coefficient a, which is the sign convention this whole module's stability analysis is built on:

a	Shape	Physical meaning
$a > 0$	centre is the *lowest* point	convex ⇒ **STABLE**
$a < 0$	centre is the *highest* point	concave ⇒ **UNSTABLE**

This matches ordinary pendulum intuition: at the rest position, if displacing the point sideways *raises* it (as on the inside of a bowl), gravity pulls it back — a restoring, stable equilibrium. If displacing it *lowers* it (as on top of a dome), the displacement is self-reinforcing — unstable.

6.3 `stability_label(a)`

```
def stability_label(a):
    if np.isnan(a):
        return "undetermined"
    return "STABLE (convex)" if a > 0 else "UNSTABLE (concave)"
```

Purpose. A thin, purely cosmetic wrapper converting a numeric curvature value into the human-readable labels used throughout the printed output and plots.

6.4 `curvature_sign_local(...)`

```
curvature_sign_local(linkage, which="L", theta_deg_range=0.3, npts=201)
```

```
def curvature_sign_local(linkage, which="L", theta_deg_range=0.3, npts=201):
    pts1 = trace_point(linkage, which, theta_deg_range=theta_deg_range, npts=
        npts)
    a1 = curvature_sign(pts1)
    pts2 = trace_point(linkage, which, theta_deg_range=theta_deg_range / 2.0,
        npts=npts)
    a2 = curvature_sign(pts2)
    if np.isnan(a1) or np.isnan(a2):
        return np.nan, False
    denom = max(abs(a1), abs(a2), 1e-300)
    converged = abs(a1 - a2) / denom < 0.05
    return a1, converged
```

Purpose. A more careful version of `curvature_sign`, intended to approximate the *true, derivative-based* curvature at $\theta = 0$ rather than a curvature averaged over some arbitrarily chosen sweep window.

A real numerical subtlety, found by direct testing. The quadratic-fit coefficient a from `curvature_sign` is *not* independent of `theta_deg_range`: for wider sweep windows, genuine higher-order (quartic and beyond) curve shape leaks into the fitted quadratic term, and a can drift — even change sign entirely — as the window widens. This was confirmed to be a real curve-shape effect, not floating-point noise: going from a 0.3° window to a 3° window changed the fitted sign outright at one tested geometry. The correct fix is to use a small, fixed window and verify convergence by halving it further, *not* to average results across several different window sizes (a different, less appropriate diagnostic that was mistakenly tried first, and which is appropriate only for a genuinely noise-dominated problem, not this curve-shape effect).

Returns. A tuple `(a, converged)`, where `a` is the curvature at the smaller window, and `converged` is `True` only if halving the window changes the result by less than 5% relative to its

own magnitude.

> This relative-convergence gate is deliberately *not* used inside `solve_q_crossing`'s root-finding objective (Section 8) — near the root itself, a is by definition close to zero, and any purely relative criterion fails spuriously there (a tiny absolute difference between two already-tiny numbers can look like a large relative disagreement). `curvature_sign_local` is best used as an external, one-off validation check (as done throughout this module's own testing), not as a per-iteration gate near a suspected root.

7 Sections 7–8 — Stability-Crossing Search

7.1 `find_q_crossing(...)`

`find_q_crossing(W, L0, q_range=(50.0, 3000.0), n=60, theta_deg_range=0.3, isosceles="apex_at_C")`

```
def find_q_crossing(W, L0, q_range=(50.0, 3000.0), n=60, theta_deg_range=0.3,
                    isosceles="apex_at_C"):
    qs = np.linspace(q_range[0], q_range[1], n)
    avals = np.empty_like(qs)
    for i, q in enumerate(qs):
        A, B, C, D, P, L = geometry_at_base_depth(W, q, L0, isosceles)
        linkage = RobertsLinkageCLD(A, B, C, D, P, L)
        a, converged = curvature_sign_local(linkage, "L", theta_deg_range)
        avals[i] = a if converged else np.nan

    valid = ~np.isnan(avals)
    qs_v, av_v = qs[valid], avals[valid]
    if len(qs_v) < 2:
        return None
    sign_changes = np.where(np.diff(np.sign(av_v)) != 0)[0]
    if len(sign_changes) == 0:
        return None
    i = sign_changes[0]
    return qs_v[i], qs_v[i + 1]
```

Purpose. Scans the base depth q (holding W and L's rest *depth* fixed) across `n` evenly-spaced samples in `q_range`, evaluates L's local curvature at each, and returns a bracket $(q_{\text{lo}}, q_{\text{hi}})$ straddling the first sign change found — suitable input for `solve_q_crossing`.

> **Why q, and not L's own depth, is the meaningful tuning parameter.** Direct scanning was tried both ways. Varying L's own rest depth (with q fixed) produces *no smooth* sign change anywhere in the physically sensible range — L stays concave continuously from near P all the way down to just above the C–D baseline, and the only sign flip found that way happens *discontinuously*, exactly where L's rest point crosses through the C–D line itself (a topological flip of the triangle C-L-D, not a meaningful tuning effect). Varying the base depth q instead, with L's rest position held fixed, gives a smooth, well-behaved sign change — confirmed numerically at $W = 265\,\text{mm}$, $L_0 = (0, -29.2)$: the crossing sits between $q = 1150$ and $q = 1300\,\text{mm}$.

Parameters.

- `W`: half-width, mm.

- `L0`: L's rest position (only y used).
- `q_range`: $(q_{\min}, q_{\max})$ to scan, mm.
- `n`: number of scan points.
- `theta_deg_range`: passed to `curvature_sign_local`.

Returns. A 2-tuple `(q_lo, q_hi)` bracketing the crossing, or `None` if no sign change is found (either because none exists in `q_range`, or because every sample failed the convergence check).

7.2 `solve_q_crossing(...)`

`solve_q_crossing(W, L0, q_bracket, theta_deg_range=0.3, isosceles="apex_at_C", xtol=1e-3)`

```
def solve_q_crossing(W, L0, q_bracket, theta_deg_range=0.3,
                     isosceles="apex_at_C", xtol=1e-3):
    from scipy.optimize import brentq

    def f(q):
        A, B, C, D, P, L = geometry_at_base_depth(W, q, L0, isosceles)
        linkage = RobertsLinkageCLD(A, B, C, D, P, L)
        pts = trace_point(linkage, "L", theta_deg_range=theta_deg_range, npts
            =201)
        return curvature_sign(pts)

    q_lo, q_hi = q_bracket
    f_lo, f_hi = f(q_lo), f(q_hi)
    if (f_lo > 0) == (f_hi > 0):
        raise ValueError(...)
    return brentq(f, q_lo, q_hi, xtol=xtol)
```

Purpose. Given a bracket known to contain a sign change (from `find_q_crossing`), root-finds the *exact* base depth q^* at which L's local curvature is precisely zero, using `scipy.optimize.brentq` (a robust, guaranteed-to-converge bracketing root-finder, given a valid sign-changing bracket).

Verified result for the default geometry ($W = 265\,\text{mm}$, $L_0 = (0, -29.2)$):

$$q^* \approx 1229.33\,\text{mm}, \qquad \frac{q^*}{W} \approx 4.64.$$

Why the objective function here does not reuse the convergence gate from `curvature_sign_local`. Near the root itself, the curvature is — by definition — close to zero, which makes the *relative* convergence test in `curvature_sign_local` fail spuriously (small absolute differences between two already-tiny numbers look like large relative disagreements). This exact pathology was hit and diagnosed directly during development: an early version of this function raised `ValueError("Curvature estimate did not converge")` even at the true root. The fix implemented here is to use a small, fixed `theta_deg_range` directly inside `brentq`'s objective (already externally validated to be well converged away from the root), without re-checking convergence on every single evaluation.

Raises `ValueError` if `q_bracket` does not actually straddle a sign change (both endpoints have the same sign).

8 Section 5 — Interactive Explorer

8.1 `interactive_explorer(theta_range_deg=10.0)`

Purpose. Opens a matplotlib window with a live linkage visualisation and four sliders:

Slider	Range	Effect
`theta`	$\pm 10^\circ$	drive angle; cheap re-draw only
`L depth`	$[-300, 50]$ mm	L's rest y-coordinate; full rebuild
`q`	$[50, 3000]$ mm	base depth of C, D; full rebuild
`W`	$[100, 500]$ mm	half-width of A, B; full rebuild

Internal structure. A nested `build(W, q, L_depth)` function calls `geometry_at_base_depth`, constructs a new `RobertsLinkageCLD`, and recomputes both the P and L traces — this full pipeline re-runs whenever `L depth`, `q`, or `W` changes, since each of those changes the fixed link lengths themselves. A separate, cheap `redraw()` function re-evaluates only `linkage.configuration(theta)` and re-draws the snapshot this alone runs when only `theta` changes, keeping that slider responsive. State is threaded through a plain `dict` (`state`) rather than globals, so the two callbacks (`rebuild` and `redraw`) share the current linkage/traces cleanly.

Live readout. The plot title always shows the current values of all four sliders plus the live curvature a and stability label for both P and L (evaluated over the full `theta_range_deg` sweep, i.e. using `curvature_sign`, not the more careful local estimator appropriate here since this is a live visual aid, not a precise root-finding calculation).

9 Section 6 — Static Figure Output

9.1 `plot_traces(theta_deg_range=5.0, save_path=None)`

Purpose. Produces a two-panel static figure using the default geometry: the left panel shows the linkage at rest (all six links, labelled A through L); the right panel shows the small-angle traces of both P and L, annotated with their RMS flatness (for P) and curvature sign/stability label (for both).

Parameters.

- `theta_deg_range`: half-width of the sweep used for both traces, degrees.
- `save_path`: if given, saves the figure to this path (creating any missing parent directory) instead of calling `plt.show()`.

Returns. The matplotlib `Figure` object, in case the caller wants to modify it further.

10 The `__main__` Menu

Running the script directly builds the default linkage, prints its derived link lengths, and offers a menu:

Option	Action
`1`	`interactive_explorer()` — opens the slider GUI.
`2`	Prints P and L trace coordinates (11 points, $\pm 5°$) plus their curvature/stability.
`3`	`plot_traces(...)`, saved to `output/P_and_L_traces.png`.
`4`	`find_q_crossing(...)` — prints the bracket around the stability crossing.
`5`	Prompts for `W` (default 265 if left blank), then finds the bracket and solves for the exact q^*.
`all`	Runs 2, 3, and (using the default W derived from $\|B - A\|/2$) the crossing search, without opening an interactive window.

11 Summary of Verified Numerical Results

Quantity	Value	Notes
$AC = BD$	287.334 mm	derived from default rest geometry
$CP = DP$	287.334 mm	
$CL = DL$	261.771 mm	
P trace RMS flatness	0.154 mm	over $\pm 5°$, default geometry
P curvature	$a \approx -2.60 \times 10^{-4}$	UNSTABLE (concave)
L curvature	$a \approx -2.69 \times 10^{-4}$	UNSTABLE (concave), matches Linkage
q^* (crossing, $W = 265$)	1229.33 mm	aspect ratio $q^*/W \approx 4.64$
q^* (crossing, $W = 300$)	1566.44 mm	aspect ratio $q^*/W \approx 5.22$